Projeto do Robô

Entrega 1 — Rota de menor custo com recursão

Grupo 06
Prof. Jemerson Damásio
Linguagem: Python 3

Integrantes
Leonardo Lucas
Carlos Adrians
Emanuel

08/11/2026

Sumário

- 1. Descrição do problema
- 2. Solução recursiva
- 3. Testes
- 4. Por que funciona
- 5. Medição do trabalho
- 6. Regra gulosa
- 7. Conferência com os critérios de avaliação
- Apêndice — Como executar e arquivos entregues

Todos os números deste relatório foram obtidos executando o código entregue (o relatório é gerado pelo script `relatorio/gerar_relatorio.py`, que chama as funções do projeto e roda os testes automáticos no momento da geração).

1. Descrição do problema

Um robô está no **canto superior esquerdo** de uma grade retangular e precisa chegar ao **canto inferior direito**. Cada casa tem um custo de energia. O objetivo é encontrar o **menor custo total** possível para fazer esse trajeto, ou informar que o trajeto é impossível.

Regras

- O robô anda **uma casa por vez**, e só para a **direita** ou para **baixo**.
- Ele **não sai da grade** e **não entra em obstáculos** (casas marcadas com X, representadas por `null` no JSON).
- O custo de uma rota é a **soma de todas as casas visitadas**, incluindo a casa de partida e a de chegada.
- Se a partida ou a chegada forem obstáculos, ou se os obstáculos bloquearem todas as rotas, não existe caminho.

Entrada

A entrada é um **texto** no formato JSON com a chave `custos`, que contém a grade como uma lista de linhas. Exemplo (a grade do enunciado):

```
{"custos": [[2, 1, 9, 9, 9],
            [3, null, 1, 1, 9],
            [1, 1, 1, null, 2],
            [9, 9, 1, 1, 1]]}
```

A entrada é **válida** somente quando:

- o texto é um JSON bem formado e é um **objeto** com a chave `custos` (outras chaves são ignoradas);
- `custos` é uma **lista não vazia** de linhas;
- cada linha é uma **lista não vazia** e todas as linhas têm o **mesmo tamanho**;
- cada valor é um **inteiro positivo** (1, 2, 3, ...) ou `null`. Não valem `true`/`false`, textos como `"1"`, números com casas decimais como `1.5`, zero ou negativos.

Saída

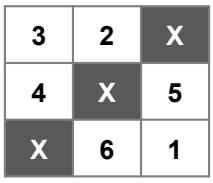
A função `resolver(entrada)` devolve um dicionário Python com um destes formatos:

Situação	Saída
Existe rota	`{"status": "ok", "custo": 11}`
Entrada válida, sem rota	`{"status": "sem_caminho", "custo": null}`
Entrada inválida	`{"status": "entrada_invalida", "erro": "mensagem"}`

Em Python o `null` aparece como `None`. O campo `caminho` (lista de coordenadas) não é incluído, pois só será exigido na Entrega 2.

Exemplos próprios do grupo

a) Entrada válida sem rota. Todos os valores são inteiros positivos ou null e as linhas têm o mesmo tamanho, então a entrada é válida. Porém, da partida (3) o robô só pode ir para o 2 (direita) ou para o 4 (baixo). Do 2, a direita e a casa de baixo são obstáculos; do 4, a casa de baixo e a da direita também são. O robô fica cercado e nunca chega ao 1 da chegada.




```
Entrada: {"custos": [[3, 2, null], [4, null, 5], [null, 6, 1]]}
Saída:   {"status": "sem_caminho", "custo": null}
```

b) Entrada inválida. A chave `custos` existe, mas o seu valor é um texto e não uma lista de linhas. Por isso a grade não pode ser montada e o programa recusa a entrada, explicando o motivo:

```
Entrada: {"custos": "1 2 3"}
Saída:   {"status": "entrada_invalida", "erro": "'custos' deve ser uma lista não vazia de linhas"}
```

2. Solução recursiva

A ideia central é responder, para uma casa qualquer, à pergunta: *"qual é o menor custo para ir desta casa até a chegada?"*. A função interna `_menor_custo(mapa, linha, coluna, contador)` faz isso:

- **Obstáculo** → devolve None (impossível).
- **Chegada** → devolve o custo da própria casa.
- **Qualquer outra casa** → toda rota que sai daqui começa indo para a direita ou para baixo. A função chama a si mesma para as duas opções (somente as que estão dentro da grade; posições fora da grade contam como impossíveis e nem são chamadas), escolhe a menor resposta válida e soma o custo da casa atual. Se nenhuma das duas chega, devolve None.

`recursivo(mapa)` apenas começa a recursão na partida (0, 0). **Não há memoização**: nada é guardado entre chamadas — se duas rotas passam pela mesma casa, a casa é calculada de novo. O objeto `Contador` guarda só *quantas* chamadas aconteceram (usado na tarefa 5), nunca um resultado.

```
def _menor_custo(mapa, linha, coluna, contador):
    """Menor custo para ir da casa (linha, coluna) até a chegada.

    Devolve None quando é impossível chegar a partir desta casa.
    """
    contador.chamadas += 1

    custo_casa = mapa[linha][coluna]

    # Caso base 1: obstáculo -> impossível.
    if custo_casa is OBSTACULO:
        return None

    ultima_linha = len(mapa) - 1
    ultima_coluna = len(mapa[0]) - 1

    # Caso base 2: chegada -> a resposta é o custo da própria casa.
    if linha == ultima_linha and coluna == ultima_coluna:
        return custo_casa

    # Caso recursivo: toda rota daqui começa indo para a direita OU para baixo.
    # Posições fora da grade contam como impossíveis, por isso só chamamos o
    # vizinho quando ele existe dentro da grade.
    melhor_resto = None

    if coluna < ultima_coluna:                      # existe casa à direita
        pela_direita = _menor_custo(mapa, linha, coluna + 1, contador)
        if pela_direita is not None:
            melhor_resto = pela_direita

    if linha < ultima_linha:                        # existe casa abaixo
        por_baixo = _menor_custo(mapa, linha + 1, coluna, contador)
        if por_baixo is not None and (melhor_resto is None or por_baixo < melhor_resto):
            melhor_resto = por_baixo

    # Nenhum dos dois caminhos chega: daqui também é impossível.
    if melhor_resto is None:
        return None

    return custo_casa + melhor_resto

def recursivo(mapa):
    """Menor custo da partida (0, 0) até a chegada, ou None se não há rota."""
    return _menor_custo(mapa, 0, 0, Contador())

def contar_chamadas(mapa):
    """Executa a mesma recursão e devolve (menor custo, número de chamadas)."""
    contador = Contador()
    custo = _menor_custo(mapa, 0, 0, contador)
    return custo, contador.chamadas
```

Validação e função de contrato

`resolver(entrada)` lê o texto com o leitor de JSON da linguagem (`json.loads`), confere as regras de validade com `validar_custos` e só então chama `recursivo`. Observação: em Python `True` também é do tipo `int`; por isso o código recusa `bool` explicitamente.

```
def validar_custos(custos):
    """Confere se `custos` é uma grade válida.

    Devolve None quando está tudo certo, ou um texto explicando o erro.
    Regras: lista não vazia de linhas; cada linha é uma lista não vazia;
    todas as linhas têm o mesmo tamanho; cada valor é inteiro positivo ou None.
    """
    if not isinstance(custos, list) or len(custos) == 0:
        return "'custos' deve ser uma lista não vazia de linhas"

    largura = len(custos[0]) if isinstance(custos[0], list) else None

    for i, linha in enumerate(custos):
        if not isinstance(linha, list) or len(linha) == 0:
            return f"a linha {i} deve ser uma lista não vazia"
        if len(linha) != largura:
            return (f"todas as linhas devem ter o mesmo tamanho "
                    f"(linha 0 tem {largura}, linha {i} tem {len(linha)})")
        for j, valor in enumerate(linha):
            if valor is OBSTACULO:
                continue
            # Em Python, True/False também são int; por isso excluímos bool.
            if isinstance(valor, bool) or not isinstance(valor, int):
                return (f"valor inválido na linha {i}, coluna {j}: "
                        f"{valor!r} (use inteiro positivo ou null)")
            if valor <= 0:
                return (f"valor inválido na linha {i}, coluna {j}: "
                        f"{valor} (o custo deve ser maior que zero)")
    return None

def resolver(entrada):
    """Recebe o TEXTO JSON da entrada e devolve um dicionário de resultado.

    {"status": "ok", "custo": <inteiro>}
    {"status": "sem_caminho", "custo": None}
    {"status": "entrada_invalida", "erro": "<mensagem>"}
    """
    if not isinstance(entrada, str):
        return {"status": "entrada_invalida", "erro": "a entrada deve ser um texto JSON"}

    try:
        dados = json.loads(entrada)
    except (ValueError, RecursionError) as erro:
        return {"status": "entrada_invalida", "erro": f"JSON mal formado: {erro}"}

    if not isinstance(dados, dict):
        return {"status": "entrada_invalida",
                "erro": "a entrada deve ser um objeto JSON com a chave 'custos'"}
    if "custos" not in dados:
        return {"status": "entrada_invalida", "erro": "falta a chave 'custos'"}

    mensagem = validar_custos(dados["custos"])
    if mensagem is not None:
        return {"status": "entrada_invalida", "erro": mensagem}

    custo = recursivo(dados["custos"])
    if custo is None:
        return {"status": "sem_caminho", "custo": None}
    return {"status": "ok", "custo": custo}
```

3. Testes

Os testes ficam em `tests/test_robo.py`, usam o módulo `unittest` (já vem com o Python) e rodam sozinhos com o comando `python -m unittest discover -s tests -v`. Cada caso compara a saída de resolver com o resultado esperado; nos casos de entrada inválida o teste confere o status e se há uma mensagem em `erro`.

Resultado da execução no momento da geração deste relatório: 15 testes executados, 0 falhas — **TODOS PASSARAM (OK)**.

3.1 Casos exigidos pelo professor

Entrada	Esperado	Obtido	
`[[7]]`	ok, custo 7	ok, custo 7	OK
`[[1, 2, 3]]`	ok, custo 6	ok, custo 6	OK
`[[1], [2], [3]]`	ok, custo 6	ok, custo 6	OK
`[[1, 1, 1], [1, 1, 1]]`	ok, custo 4	ok, custo 4	OK
`Grade do exemplo`	ok, custo 11	ok, custo 11	OK
`[[1, 1, null], [2, null, 1], [1, 1, 1]]`	ok, custo 6	ok, custo 6	OK
`[[null]]`	sem_caminho	sem_caminho	OK
`[[null, 1], [1, 1]]`	sem_caminho	sem_caminho	OK
`[[1, 1], [1, null]]`	sem_caminho	sem_caminho	OK
`[[1, null], [null, 1]]`	sem_caminho	sem_caminho	OK
`custos = []`	entrada_invalida	entrada_invalida	OK
`custos = [[]]`	entrada_invalida	entrada_invalida	OK
`custos = [[1, 2], [3]]`	entrada_invalida	entrada_invalida	OK
`custos = [[true, 1]]`	entrada_invalida	entrada_invalida	OK
`custos = [["1", 1]]`	entrada_invalida	entrada_invalida	OK
`custos = [[1.5, 1]]`	entrada_invalida	entrada_invalida	OK
`custos = [[0, 1]]`	entrada_invalida	entrada_invalida	OK
`custos = [[-1, 1]]`	entrada_invalida	entrada_invalida	OK
`Texto {"matriz": [[1]]}`	entrada_invalida	entrada_invalida	OK
`Texto [[1, 2]]`	entrada_invalida	entrada_invalida	OK
`Texto {"custos": [[1, 2]`	entrada_invalida	entrada_invalida	OK

3.2 Casos criados pelo grupo

Entrada	Esperado	Obtido	
`Próprio: [[3, 2, null], [4, null, 5], [null, 6, 1]]`	sem_caminho	sem_caminho	OK
`Próprio: {"custos": "1 2 3"}`	entrada_invalida	entrada_invalida	OK
`Próprio: [[1, 3, 1], [1, 5, 1], [4, 2, 1]]`	ok, custo 7	ok, custo 7	OK
`Próprio: [[1, 1, 1], [null, null, null], [1, 1, 1]]`	sem_caminho	sem_caminho	OK
`Próprio: [[1, 1, 1], [2, 9, 9], [1, 1, 1]]`	ok, custo 6	ok, custo 6	OK
`Próprio: [[null, 1, 1], [1, 1, 1]]`	sem_caminho	sem_caminho	OK
`Próprio: {"custos": null}`	entrada_invalida	entrada_invalida	OK

Entrada	Esperado	Obtido	
`Próprio: texto vazio`	entrada_invalida	entrada_invalida	**OK**

3.3 Outros testes automáticos

- contagem de chamadas nas grades 2×3, 3×3, 6×6, 10×10 e 1×10 e na grade do exemplo (tarefa 5);
- regra gulosa: 26 contra 11 na grade do exemplo, grade própria em que ela erra, grade própria em que ela fica presa, empate preferindo a direita, partida bloqueada e grade de uma casa (tarefa 6);
- recursivo não altera a grade recebida; saída não tem o campo caminho; entrada que não é texto é recusada.

3.4 Saída do executor de testes

```
test_empate_prefere_a_direita (test_robo.TestGuloso.test_empate_prefere_a_direita) ... ok
test_grade_de_uma_casa (test_robo.TestGuloso.test_grade_de_uma_casa) ... ok
test_grade_do_exemplo_da_26_contra_11 (test_robo.TestGuloso.test_grade_do_exemplo_da_26_contra_11) ... ok
test_grade_propria_em_que_o_guloso_erra (test_robo.TestGuloso.test_grade_propria_em_que_o_guloso_erra) ... ok
test_grade_propria_em_que_o_guloso_fica_preso (test_robo.TestGuloso.test_grade_propria_em_que_o_guloso_fica_preso) ... ok
test_partida_bloqueada (test_robo.TestGuloso.test_partida_bloqueada) ... ok
test_chamadas_na_grade_do_exemplo (test_robo.TestMedicao.test_chamadas_na_grade_do_exemplo) ... ok
test_chamadas_nas_grades_de_uns (test_robo.TestMedicao.test_chamadas_nas_grades_de_uns) ... ok
test_grade_do_exemplo (test_robo.TestRecursivo.test_grade_do_exemplo) ... ok
test_nao_altera_a_grade (test_robo.TestRecursivo.test_nao_altera_a_grade) ... ok
test_sem_rota_devolve_none (test_robo.TestRecursivo.test_sem_rota_devolve_none) ... ok
test_casos_do_grupo (test_robo.TestResolver.test_casos_do_grupo) ... ok
test_casos_do_professor (test_robo.TestResolver.test_casos_do_professor) ... ok
test_entrada_que_nao_e_texto (test_robo.TestResolver.test_entrada_que_nao_e_texto) ... ok
test_saida_sem_campo_caminho (test_robo.TestResolver.test_saida_sem_campo_caminho) ... ok

----------------------------------------------------------------------
Ran 15 tests in 0.043s

OK
```

4. Por que funciona

Por que a recursão sempre termina?

De uma casa (linha, coluna) até a chegada faltam exatamente (última linha − linha) + (última coluna − coluna) movimentos. Cada chamada recursiva anda uma casa para a direita ou para baixo, então **a cada chamada falta um movimento a menos**. Esse número nunca fica negativo, porque só chamamos vizinhos que estão dentro da grade. Logo, depois de no máximo linhas + colunas − 2 chamadas encadeadas, chega-se à chegada (caso base) ou a um obstáculo (outro caso base), e a recursão para. Na pior das hipóteses a profundidade é linhas + colunas − 1.

Por que ela encontra o menor custo?

Toda rota que sai de uma casa (que não é a chegada) **começa indo para a direita ou para baixo** — não há outra opção. Então o menor custo a partir dela é o custo da casa mais o menor entre: o melhor custo a partir da casa da direita e o melhor custo a partir da casa de baixo. Supondo que as chamadas para os vizinhos (que estão mais perto da chegada) já devolvem o menor custo correto, escolher o menor dos dois dá o menor custo correto para a casa atual. Como o caso base (chegada = custo da própria casa) está certo, por indução a resposta está certa em todas as casas, inclusive na partida. Em outras palavras: a recursão examina, de forma implícita, **todas** as rotas possíveis e fica com a mais barata.

O que acontece quando não há rota?

Obstáculos devolvem None, e posições fora da grade nunca são escolhidas. Uma casa só devolve um número quando pelo menos um dos vizinhos devolveu um número; se os dois devolveram None (ou não existem), ela também devolve None. Assim o None “sobe” até a partida exatamente quando nenhuma rota consegue chegar. `resolver` transforma esse None em `{"status": "sem_caminho", "custo": null}`. Exemplos: [[null]] (partida bloqueada), [[1, 1], [1, null]] (chegada bloqueada) e [[1, null], [null, 1]] (obstáculos no meio).

5. Medição do trabalho

Usamos `contar_chamadas(mapa)`, que executa **exatamente a mesma recursão** e conta quantas vezes a função foi chamada (incluindo a primeira chamada, na partida). Como o programa só chama vizinhos que estão dentro da grade, os números batem com a tabela do enunciado:

Grade (casas = 1)	Enunciado	Medido	C(L+C, L) − 1	Confere?
2 × 3	9	9	9	sim
3 × 3	19	19	19	sim
6 × 6	923	923	923	sim
10 × 10	184.755	184.755	184.755	sim
1 × 10	10	10	10	sim
Grade do exemplo (4 × 5)	—	43	125 sem obstáculos	—

Explicação dos números

Sem memoização, a função é chamada em uma casa **uma vez para cada rota diferente que leva da partida até ela**. Até a casa (i, j) existem C(i + j, i) rotas (escolher quais dos i + j passos são para baixo). Somando em todas as casas de uma grade L × C, o total de chamadas é **C(L + C, L) − 1**, coluna que calculamos acima e que coincide com a medição.

- **1 × 10**: só existe uma rota, então cada casa é chamada uma única vez: 10 chamadas.
- **2 × 3**: as casas recebem 1, 1, 1 / 1, 2, 3 chamadas = 9.
- **10 × 10**: 184.755 chamadas para uma grade de apenas 100 casas. Só a casa de chegada é chamada C(18. 9) = 48.620 vezes. O trabalho cresce de forma exponencial, porque o mesmo pedaço da grade é recalculado muitas vezes. (Guardar esses resultados seria a memoização, que fica para a Entrega 2.)

Grade do exemplo

Na grade do exemplo foram medidas **43 chamadas** (custo 11). Sem obstáculos, uma grade 4 × 5 teria 125. A diferença acontece porque os dois obstáculos também recebem chamadas, mas param imediatamente (caso base) e não geram chamadas para a frente; todas as rotas que passariam por eles são cortadas. A tabela abaixo mostra quantas vezes cada casa foi chamada (obstáculos em cinza). A soma é 43, igual à medição, e as 7 chamadas na chegada correspondem às 7 rotas que existem nessa grade.

2	1	9	9	9
3	X	1	1	9
1	1	1	X	2
9	9	1	1	1

Custos

1	1	1	1	1
1	2	1	2	3
1	1	2	4	3
1	2	4	4	7

Chamadas por casa

6. Regra gulosa

`guloso(mapa)` começa na partida e, a cada passo, olha apenas os vizinhos **livres** (direita e baixo, dentro da grade e sem obstáculo) e anda para o **mais barato**. No **empate prefere a direita** (comparação `direita <= baixo`). Se não houver vizinho livre antes da chegada, o robô fica preso e a função devolve None. Ela nunca volta atrás nem olha o futuro — por isso pode errar.

```
def guloso(mapa):
    """Anda sempre para o vizinho livre mais barato (direita no empate).

    Devolve o custo total da rota escolhida, ou None se a partida for
    obstáculo ou se o robô ficar preso (sem vizinho livre) antes da chegada.
    """
    ultima_linha = len(mapa) - 1
    ultima_coluna = len(mapa[0]) - 1

    linha, coluna = 0, 0
    if mapa[0][0] is OBSTACULO:
        return None
    total = mapa[0][0]

    while not (linha == ultima_linha and coluna == ultima_coluna):
        direita = None
        baixo = None
        if coluna < ultima_coluna and mapa[linha][coluna + 1] is not OBSTACULO:
            direita = mapa[linha][coluna + 1]
        if linha < ultima_linha and mapa[linha + 1][coluna] is not OBSTACULO:
            baixo = mapa[linha + 1][coluna]

        if direita is None and baixo is None:
            return None                     # ficou preso: a regra não chega

        # "<=" faz a direita ganhar no empate.
        if direita is not None and (baixo is None or direita <= baixo):
            coluna += 1
            total += direita
        else:
            linha += 1
            total += baixo

    return total
```

6.1 Grade do exemplo: 26 contra 11

Resultado medido: **guloso = 26**, **recursivo = 11**.

2	1	9	9	9
3	X	1	1	9
1	1	1	X	2
9	9	1	1	1

Rota gulosa: 2+1+9+1+1+9+2+1 = 26

2	1	9	9	9
3	X	1	1	9
1	1	1	X	2
9	9	1	1	1

Rota ótima: 2+3+1+1+1+1+1+1 = 11

Onde a escolha atrapalhou: logo no primeiro passo o guloso compara 1 (direita) com 3 (baixo) e vai para a direita. Parece mais barato, mas leva à linha de cima, onde a casa de baixo do 1 é obstáculo e a única saída é um 9. Depois, na casa (1, 2), há empate entre 1 (direita) e 1 (baixo); a preferência pela direita leva a outro 9. A rota ótima aceita pagar 3 no início para descer pela primeira coluna e seguir só por casas de custo 1.

6.2 Grade própria (3 × 3) em que o guloso erra

```
{"custos": [[1, 1, 1],
            [2, 9, 9],
            [1, 1, 1]]}
```

Resultado medido: **guloso = 13**, **recursivo = 6**.

1	1	1
2	9	9
1	1	1

Rota gulosa: 1+1+1+9+1 = 13

1	1	1
2	9	9
1	1	1

Rota ótima: 1+2+1+1+1 = 6

Onde a escolha atrapalhou: na partida o guloso compara 1 (direita) com 2 (baixo) e vai para a direita para “economizar” 1. Ao fazer isso, ele fica do lado de cima de uma parede de 9s: toda rota que começa pela direita precisa atravessar a linha do meio por um 9. Quem desce primeiro paga 2, mas passa pela única casa barata da linha do meio e termina em 6. Uma economia de 1 no início custou 7 a mais no total.

6.3 Grade própria (3 × 3) em que o guloso fica preso

```
{"custos": [[1, 1, 1],
            [5, 1, null],
            [5, 5, 1]]}
```

Resultado medido: **guloso = sem caminho (não chega)**, **recursivo = 9**.

1	1	1
5	1	X
5	5	1

Rota gulosa: fica presa no canto (0, 2)

1	1	1
5	1	X
5	5	1

Rota ótima: 1+1+1+5+1 = 9

Onde a escolha atrapalhou: na casa (0, 1) há empate entre 1 (direita) e 1 (baixo). A regra prefere a direita e vai para o canto (0, 2), onde a direita sai da grade e a casa de baixo é obstáculo: o robô fica preso, mesmo existindo uma rota de custo 9. A regra gulosa nem sempre encontra uma rota.

7. Conferência com os critérios de avaliação

Critério (pontos)	Onde está atendido
Descrição do problema (0,5)	Seção 1: entrada, saída, objetivo, regras e regras de validade; exemplo próprio de entrada válida sem rota e de entrada inválida, com a saída real do programa.
Recursão e testes (1,0)	Seção 2: recursivo sem memoização; Seção 3: 21 casos do professor e 8 casos do grupo; 15 testes automáticos, todos passando.
Por que funciona (0,6)	Seção 4: por que termina (falta um movimento a menos), por que acerta (toda rota começa pela direita ou por baixo + indução) e como detecta a falta de rota.
Medição do trabalho (0,6)	Seção 5: chamadas medidas 9, 19, 923, 184.755, 10 (iguais ao enunciado) e 43 na grade do exemplo, com fórmula e explicação por casa.
Regra gulosa (0,3)	Seção 6: guloso com empate para a direita; 26 contra 11 no exemplo; grade própria 3 × 3 com custos (13 contra 6) e explicação de onde errou; extra: grade em que fica preso.

Fora do escopo desta entrega (de propósito): memoização, tabela de resultados e o campo `caminho` com a lista de coordenadas, que ficam para a Entrega 2.

Apêndice — Como executar e arquivos entregues

Requer Python 3.10 ou mais novo; o código e os testes não usam bibliotecas externas (a biblioteca reportlab é usada só para gerar este PDF).

```
python main.py                              # demonstração
python main.py "{\"custos\": [[1, 2], [3, 4]]}"
python -m unittest discover -s tests -v     # testes
python relatorio/gerar_relatorio.py         # gera este PDF
python empacotar.py                         # gera o .zip
```

Arquivo	Conteúdo
`projeto_robo/robo.py`	resolver, recursivo, guloso, contar_chamadas, validar_custos
`tests/test_robo.py`	testes automáticos (unittest)
`main.py`	demonstração pela linha de comando
`relatorio/gerar_relatorio.py`	gera este relatório executando o código
`empacotar.py`	gera entrega/entrega1_grupo06.zip
`README.md`	instruções e explicação do código para a apresentação